Midterm Answer Key

Data Science, AI & Emerging Paradigms

BSIT — Data Science

Concise, To-the-Point Answers Extracted from Course Handouts

Q-1. How do bagging, boosting, and stacking differ? **(05 marks)**

Bagging, boosting, and stacking are the three main **ensemble** techniques that combine multiple models to improve performance.

Aspect	Bagging	Boosting	Stacking
Core idea	Train models in parallel on varied bootstrap samples	Train models sequentially; each focuses on earlier errors	Combine outputs of several models using a meta-model
How models train	Independently, in parallel	Dependently, one after another	Base models trained, then a meta-learner on their outputs
Data used	Random subsets (with replacement)	Full data, reweighting hard cases	Predictions of base models as new features
Main effect	Reduces variance	Reduces bias (and variance)	Reduces both by blending diverse models
Combination	Majority vote / averaging (equal weight)	Weighted by model performance	Learned by the meta-model
Overfitting	Less prone to overfitting	Can overfit if not regularized	Depends on base/meta models
Examples	Random Forest	AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost	Stacked generalization with a meta-learner

Q-2. Compare Generative AI vs. Agentic AI. **(05 marks)**

Aspect	Generative AI	Agentic AI
Primary focus	Content generation	Goal-driven action
Core capability	Generate text, images, code, or audio	Decide, plan, and act
Autonomy	Often limited	Higher and more persistent
Interaction style	Prompt-based	Environment-based or tool-based
Memory	Often short-term	Often persistent or state-aware
Typical output	Content	Actions, plans, decisions, and content

Integration Principle

Generative AI often acts as the cognitive engine inside an agentic system, while agentic AI adds planning logic, tools, memory, and goal management around that engine.

Q-3. Main purposes of GitHub Copilot Ask, Edit, and Agent modes. (05 marks)

Mode	What It Does	Typical Use
Ask mode	Answers questions, explains code, suggests approaches	Learn concepts, debug errors, understand algorithms, ask for examples
Edit mode	Applies requested changes to code or text	Refactor functions, update notebooks, fix formatting, rewrite blocks
Agent mode	Works through a multi-step task using repo context and tools	Search files, update several places, run validations, complete coding tasks

How to Choose

Use **Ask** to understand topics before copying code, **Edit** when you already know what you want changed, and **Agent** for larger tasks that involve multiple steps or files.

Q-4. Compare AI Agent vs. Chatbot vs. Traditional Automation. (05 marks)

System	Main Behavior	Strength	Limitation
Basic chatbot	Answers direct questions	Easy to use	Usually limited to one-turn responses
Traditional automation	Follows fixed rules or scripts	Reliable for repeated tasks	Cannot adapt well to new situations
AI agent	Plans, acts, checks results, and continues	More flexible and capable	Needs guardrails and monitoring

Key Difference

A chatbot mainly **responds**. An AI agent can **respond, decide, act, observe, and continue working**.

Q-5. How does a data scientist proceed from a real problem to a data science task? **(05 marks)**

A real-world goal is **translated (formulated) into a concrete data science task** with a clear target and a measurable success metric, then constraints are balanced.

Real-World Goal	Data Science Formulation	Possible Metrics
Identify at-risk students	Binary classification	Recall, precision, F1-score, calibration
Forecast course enrollment	Time series forecasting	MAE, RMSE, MAPE
Recommend learning materials	Ranking or recommendation	Precision@k, Recall@k, NDCG
Detect unusual student system behavior	Anomaly detection	Detection rate, false alarm rate

Then define success criteria and tradeoffs:

- **Offline metrics:** Accuracy, RMSE, F1-score, AUC, ranking metrics.
- **Operational metrics:** Latency, throughput, availability, compute cost.
- **Business/institutional metrics:** Retention rate, intervention success, decision quality.
- **Safety and governance metrics:** Bias, drift, privacy compliance, reviewability.

Why It Matters

Real systems rarely optimize only one thing. If the project question, metric, user, or decision context is unclear, even a technically strong model may create little value.

Q-6. Explain when deep learning is a good choice. **(05 marks)**

Deep Learning Often Fits Well	Traditional ML May Be Better First
Large image, audio, or text datasets	Small datasets with limited samples
Very complex relationships in the data	Problems where high interpretability is required
Tasks where automatic feature learning helps	Cases where a simple model already works well
Applications that can use GPU acceleration	Projects with very limited computing resources

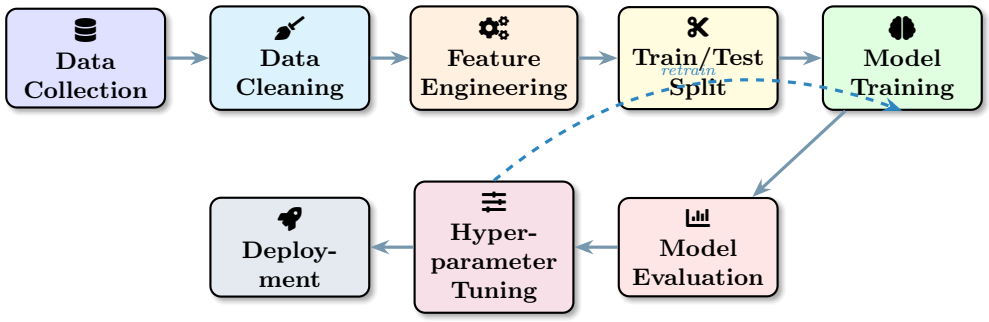
Important Note

Deep learning is not always the best first model. In many business and classroom projects, a simpler method such as logistic regression, decision trees, or random forests may be easier to train, explain, and deploy.

Q-7. What is the difference between a parameter and a hyperparameter? (05 marks)

Aspect	Parameter	Hyperparameter
Definition	A value **learned during training**	A value **chosen before training**
Who sets it	Learned automatically by the model	Set by the data scientist / engineer
Examples	Weights and biases	Learning rate, batch size, number of epochs
When fixed	Updated by the optimizer each step	Fixed for a training run; found by tuning
Role	Defines the learned model	Controls how the model learns

Q-8. Draw a diagram that shows the Machine Learning Pipeline. (05 marks)



Pipeline stages: Data Collection → Data Cleaning & Preprocessing → Feature Engineering → Train/Test Split → Model Training → Model Evaluation → Hyperparameter Tuning → Deployment, with a feedback loop to retrain.

Q-9. How different data types are prepared for deep learning. (05 marks)

Data Type	Representation	Common Preparation Steps
Images	Pixel grids	Resize, normalize pixel values, augment with flips or rotations
Text	Tokens or embeddings	Clean text, tokenize, pad sequences, build embeddings
Audio	Waveforms or spectrograms	Trim noise, convert to features, standardize length
Tabular data	Rows and columns	Handle missing values, encode categories, scale numeric features

Garbage In, Garbage Out

Even a powerful deep learning model cannot rescue poor data. Clean, relevant, and well-prepared data is still the foundation of good performance.

Q-10. GitHub Copilot prompts for a complete data science workflow. (05 marks)

The prompts below follow the workflow from **dataset creation to model evaluation**, mapped to the machine learning pipeline. Each can be given to Copilot (Ask/Edit/Agent mode), then the output must be reviewed and tested.

1. **Dataset creation:** "Generate a Python script that creates a synthetic student-performance dataset with features (study hours, attendance, prior grade) and a binary target `at_risk`, and save it to `students.csv`."
2. **Load & explore:** "Write pandas code to load `students.csv`, show `head()`, `info()`, `describe()`, and report missing values per column."
3. **Data cleaning:** "Add code to handle missing values (median for numeric, mode for categorical), remove duplicates, and fix data types."
4. **Visualization (EDA):** "Generate matplotlib/seaborn code for a correlation heatmap and histograms of all numeric features."
5. **Feature engineering:** "Write code to one-hot encode categorical columns and standard-scale numeric features using a scikit-learn `ColumnTransformer`."
6. **Train/test split:** "Add code to split the data into 80% train and 20% test with `train_test_split`, stratified on the target, `random_state=42`."
7. **Model training:** "Train a logistic regression and a random forest classifier inside a scikit-learn `Pipeline` on the training data."
8. **Model evaluation:** "Write code to evaluate both models on the test set: accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC."
9. **Hyperparameter tuning:** "Use `GridSearchCV` with 5-fold cross-validation to tune the random forest (`n_estimators`, `max_depth`) and report the best parameters."
10. **Save & explain:** "Save the best model with joblib and write a short comment explaining each pipeline step and the chosen evaluation metrics."

Reminder

GitHub Copilot suggestions must always be reviewed, tested, and understood. Generated code can still contain errors, bias, or insecure practices — the user remains responsible for correctness.